

Can the score follow the stat taps?

Options for driving the **Us / Them** rally log from Serve In, Serve Out, Return In and Return Out · CoachIQ stats client · 7 October 2026

1. The question

On the Games screen every player has four counters: Serve In, Serve Out, Return In and Return Out. The score sits apart from them: the coach taps **Us** or **Them** after each rally. The question is whether the score could be worked out from the stat taps instead, given three facts about what is and is not recorded:

1. A **Serve In** by one of our players can still lose the point.
2. A **Serve Out** by one of our players always gives the point to Them.
3. Nothing is tracked about the other team: whether they served into the net or got it over, or which side won a rally.

Short answer

More is possible than it looks. The two "Out" taps score themselves, and in rally scoring the side that wins a rally serves the next one, so the coach's *own next tap* reveals who won an open rally. Only three things stay out of reach without tracking the other team: the last rally of a set, two untracked points in a row, and a rally the coach never tapped. The Us / Them buttons stay for exactly those cases. Option D below gets the score for nothing extra in the common case; option B is the safe, small step if D feels like too much magic.

Operating assumption, confirmed: the coach taps a stat for **every** one of our serves and every one of our receptions, so each rally that starts on our side has exactly one tap.

2. What the app already knows

- **The score is already a rally log, not a counter.** Each Us / Them tap appends one letter (**U** or **T**) to `set.points`. The displayed score is a tally of that string.
- **The app already asks who served first.** The "We serve first / They serve first" bar fills `set.servedFirst` before any rally can be logged.
- **Stat taps and rally taps share one ordered history** per game. Undo pops the last entry whichever kind it is. So the sequence of taps is known, even though the counters themselves are unordered.
- **Export is the v4 contract** (per-player counts, `points`, `servedFirst`). None of the options below change what is exported.

Put the first two together: with the serve-first answer and the log so far, the app always knows **who is serving**. Whoever won the last rally serves the next one.

3. The serving-side rule

Because the server is known, every stat tap says two things: something about the rally it belongs to, and something about the rally *before* it.

Our tap	About this rally	About the previous rally
Serve Out	Them +1, immediately. Certain.	We were serving, so we won the one before (or this is the first rally).
Return Out	Them +1, immediately. Certain.	They were serving, so they won the one before.
Serve In	Outcome unknown until the next tap. The rally stays <i>open</i> .	We were serving.
Return In	Outcome unknown until the next tap. The rally stays <i>open</i> .	They were serving.
Next tap is a Serve while a rally is open	We are serving again, so we won the open rally. Log U .	
Next tap is a Return while a rally is open	They are serving, so they won the open rally. Log T .	
A tap that disagrees with the log's server (no rally open)	We tap a Serve while the log says Them serve, or a Return while it says we serve. A point we never logged happened in between: a side-out. Log one letter for it (U for the first case, T for the second).	

That third highlighted row is what covers condition 3. When their server puts the ball into the net, nobody on our bench taps anything, but our next tap is a **Serve** while the log still says they were serving. The app can insert the missing **U** at that moment.

What no our-side-only scheme can see

- **The last rally of a set.** Nothing follows it, so nothing resolves it. One Us / Them tap closes the set.
- **Two or more untracked points on the same side in a row.** Their serve error (Us +1) followed by our serve that nobody tapped and that we lost (Them +1) leaves the server where the log expects it. The app sees nothing wrong and the score runs two short. The coach taps Us, then Them.
- **A rally the coach simply never tapped** (same mechanism, one short).

4. The options

A. Keep it manual today

Stat taps and rally taps stay independent. The coach taps a counter and then Us or Them.

Taps per rally	2 for a rally that starts on our side (stat + score), 1 for an untracked one (score only).
Gets right	Everything the coach remembers to tap. No surprises, no inference.
Misses	Nothing by design, but the score and the sheet can drift apart silently when a pill tap is forgotten, and there is no way to see that they disagree.
UI change	None.
Data / undo / export	None.
Effort	None.

B. Deterministic only small step

Serve Out and Return Out append **T** automatically. Serve In and Return In do nothing to the score. Us / Them stay for every rally that is not an Out.

Taps per rally	1 for an Out rally, 2 for an In rally, 1 for an untracked one.
Gets right	Condition 2 exactly. The two most error-prone moments on the bench (an error by our player, then remembering to give Them the point) become one tap.
Misses	Conditions 1 and 3 entirely. About half the rallies still need a pill tap.
Risk	Double counting: the coach taps Return Out and then, from habit, Them. Needs a visible flash on the Them pill with a small "+1 auto" tag, and an Undo that reverses the tap and its point together.
UI change	Flash on the Them pill; Undo label "↩ Undo Brynn R out + Them".
Data / undo / export	The history entry for a count tap records the letter it appended, so one Undo reverses both. Export unchanged.
Effort	Small. Half a day with tests.

C. Pending-rally prompt B plus guidance

Everything in B. In addition, after a Serve In or Return In the rally is marked *open*: the Us / Them pills change to **Won?** **Lost?** and pulse until one is tapped. The next stat tap is still allowed (the coach is not blocked), it just leaves the earlier rally unresolved.

Taps per rally	Same as B.
Gets right	The score can no longer drift from the sheet without the coach seeing it: an open rally is visible on screen.
Misses	Conditions 1 and 3 still need a finger. No tap is saved over B.
UI change	Pill labels and a pulse while a rally is open. This is also the UI layer D needs, so building C is not wasted if D follows.
Data / undo / export	As B, plus a small per-set <code>pending</code> field (<code>null</code> , <code>'serve'</code> or <code>'return'</code>). Export unchanged.
Effort	Small to medium. About a day.

D. Side-out inference recommended

Everything in B and C, and the open rally is resolved by the **next stat tap** using the serving-side rule in section 3: a Serve tap resolves it **U**, a Return tap resolves it **T**. When no rally is open but the tap disagrees with who the log says is serving, one letter is inserted for the untracked side-out. The first stat tap of a set answers the serve-first question (a Serve means we served first), still correctable from the menu's flip action. Us / Them remain for set point and for untracked runs.

Taps per rally	1 for every rally that starts on our side. 0 for a single untracked side-out. 1 pill tap at set point and for each extra untracked point.
Gets right	Conditions 1, 2 and the common form of 3 (their serve error followed by our serve).
Misses	The three cases in the warning box above. The score also lags the sheet by one rally while a rally is open; the open-rally marker from C makes that visible.
Risk	Surprise. A point appears on a stat tap, not a pill tap. If the coach taps a stat and then, from habit, the pill for the same rally, that pill tap becomes an extra rally. Mitigations: the toast ("Us +1 · side-out"), the "auto" mark on inferred letters in the log, the Undo label naming both things it will reverse, and tinting the active column pair (Serve while we serve, Return while they serve) so a tap on the dimmed pair visibly means "a point you did not log happened".
UI change	Pending pills from C; toast above the bottom bar; "serving: Us / Them" chip; dimmed inactive column pair; richer Undo label.
Data / undo / export	Count history entries carry the letters they appended (zero, one or two) and whether they answered serve-first, so one Undo reverses all of it. New <code>pending</code> field per set. Session schema bump to 5 with a trivial migration (<code>pending: null</code>). Export unchanged, still v4.
Effort	Medium. One to two days including unit tests on the pure inference function and browser tests for Undo and the toast.

While they serve (log ends in T)

Us 1

3 Them

serving: Them

Brynn

0

0

2

1

Them +1 · Return Out

While a rally is open (Grace just served in)

Won?

Lost?

serving: Us

Grace

4

0

0

0

Undo: ↶ Undo Grace S in + Us

E. Track the opponent too out of scope

Add a "Them" row with Serve In / Serve Out (or a single "Their error" button). Every rally then starts with a tap on one side or the other, which closes the untracked-side-out gap. It still cannot resolve an In rally: that needs the rally's *ending* (kill, attack error, block, dig error), which is a different stat sheet with several more buttons per player. It is the only route to a fully automatic score, and the cost is a new contract version plus a busier screen that no longer fits a full roster. Noted so the ceiling is clear; not recommended.

5. Side by side

	A manual	B deterministic	C prompt	D inference	E opponent row
Taps for an Out rally	2	1	1	1	1
Taps for an In rally	2	2	2	1	2
Taps for their serve error	1	1	1	0 (usually)	1
Condition 1 (Serve In can lose)	manual	manual	manual, prompted	inferred	manual
Condition 2 (Serve Out = Them)	manual	auto	auto	auto	auto
Condition 3 (their side untracked)	manual	manual	manual	single side-outs inferred	tracked
Score can drift unseen	yes	yes	no	no	no
Double-count risk	—	low, after Outs	low	medium, needs feedback	low
Score lags the sheet	no	no	no	by one open rally	no
Undo	as now	tap + letter	tap + letter	tap + letters + serve-first	new row
Export contract	v4	v4	v4	v4	v5
Effort	none	½ day	1 day	1–2 days	3+ days

Worked-example tap totals (section 6, ten rallies): A 19 taps · B 17 · C 17 · D 10 · E 18 plus the opponent row's taps.

6. Worked example

Ten rallies of set 1, we serve first. Column D shows the log and the score *as the coach would see it* after each tap; inferred letters are underlined in blue, an open rally is the dashed box.

#	What happened	Tap	True score	D: log after the tap	D shows	A / B pill taps
1	Grace serves in, we win the rally	Grace Serve In	1–0	? serve-first answered: Us	0–0	Us / Us
2	Grace serves in again, we lose	Grace Serve In	1–1	U ?	1–0	Them / Them
3	They serve, Addison returns in, we lose	Addison Return In	1–2	U T ?	1–1	Them / Them

4	They serve, Brynn's return goes out	Brynn Return Out	1-3	<u>U</u> <u>T</u> <u>T</u> <u>T</u>	1-3	Them / —
5	They serve into the net. Nobody taps.	—	2-3	unchanged	1-3	Us / Us
6	Emily serves in, we win	Emily Serve In	3-3	<u>U</u> <u>T</u> <u>T</u> <u>T</u> <u>U</u> <u>?</u> the U is rally 5, inserted because the log said Them were serving	2-3	Us / Us
7	Emily serves out	Emily Serve Out	3-4	<u>U</u> <u>T</u> <u>T</u> <u>T</u> <u>U</u> <u>U</u> <u>T</u>	3-4	Them / —
8	They serve, Hailey returns in, we win	Hailey Return In	4-4	unchanged, plus <u>?</u>	3-4	Us / Us
9	Brooklyn serves in, we win	Brooklyn Serve In	5-4	... <u>T</u> <u>U</u> <u>?</u>	4-4	Us / Us
10	Brooklyn serves in, we lose. Set over.	Brooklyn Serve In	5-5	... <u>T</u> <u>U</u> <u>U</u> <u>?</u>	5-4	Them / Them
	Set point: nothing follows, so the coach closes it	Them pill	5-5	U T T T U U T U U T	5-5	

Blue underline: letter came from a stat tap. Dashed box: open rally, resolved by the next tap. "A / B pill taps": what the coach must tap in addition under option A and under option B.

Totals for these ten rallies: option A needs 9 stat taps and 10 pill taps; B and C need 9 and 8; D needs 9 and 1. Every option ends on the same log, UTTTUUTUUT, which is what gets exported.

7. Edge cases for D

Case	Behaviour
Set point	The last rally stays open until the coach taps Us or Them. The open-rally pills make this obvious. Leaving it open is harmless: the exported log is simply one rally short, same as forgetting a pill tap today.
Two untracked points in a row	Not detectable. The coach taps the pills for them, in order. If they are noticed late, the pills still append at the end; the tally is right even if the order inside the log is not.
Untouched ace against us	If the coach taps Return Out for the receiver, Them + 1 is logged. If nobody taps, it is an untracked point like any other.
Coach taps the pill for a rally the app already inferred	An extra rally is logged. The toast and the auto mark exist to make this rare; Undo fixes it with one tap because the pill tap is the top of the history.
Minus mode	A decrement only reverses the count. It never adds or removes letters and never changes the open rally. The right tool for a mis-tap is still Undo, which reverses the tap and everything it inferred.
Undo	One Undo reverses the count, the zero-to-two letters that tap appended, and the serve-first answer if that tap gave it. The label says so: "↶ Undo Emily S out + Us, Them". Undoing a pill tap re-opens the rally it closed.
Serve-first	Answered by the first stat tap of the set. Wrong when their first serve was an error before our first tap; the existing flip action in the menu corrects it, and the tally is one short until a pill tap adds the rally.
Typed score	A set with a typed score has no log, so no inference. Same exclusivity rule as the pills today.
Switching sets with a rally open	The open rally belongs to its set and waits there. Each set has its own <code>pending</code> .
Rally cap (200) and history cap (400)	At the cap the letter is dropped and the count still increments, as the pill does today. A count tap with letters is one history entry, so the 400 cap is unchanged in practice.
Export	Unchanged. The planner receives the same v4 payload; it cannot tell an inferred letter from a tapped one.
Reload mid-set	<code>pending</code> is saved with the set, so an open rally survives a reload and a reopened tab.

8. Recommendation

Build D, in two steps that each ship on their own.

1. **B + C first:** Out taps score themselves, open rallies show "Won? / Lost?", one Undo reverses a tap and its letter. Low risk, and the open-rally state is the hook D needs.

2. **Then the inference:** resolve the open rally on the next stat tap, insert the untracked side-out, auto-answer serve-first, add the toast, the auto mark and the column tint.

If the inference ever feels wrong on the bench, step 1 stands on its own and the pills still work exactly as today.

What the work looks like if D goes ahead (a separate plan)

- **session.js:** a pure `inferPoints(set, stat, side)` that returns the letters to append and whether serve-first was answered; `tap()` uses it and writes `{ kind:'count', ..., points, servedFirstSet }`; `undo()` reverses letters and serve-first from the entry; `tapPoint()` clears `pending`; new `pending` field on the set, schema 5 with a one-line migration.
- **ui.js:** pill labels while a rally is open; toast; "serving" chip; dimmed column pair; Undo label naming the letters.
- **Tests:** the worked example above as one unit test on the log; Undo of a two-letter tap; serve-first auto-answer and its undo; minus mode leaving the log alone; a browser test that taps the sequence and reads the score.

```
on stat tap (stat, side), minus mode off:
  added = ''
  if servedFirst is null:          servedFirst = (stat == 'serve'); sfSet = true
  if pending is not null:          added += (stat == 'serve') ? 'U' : 'T'      // resolve the
open rally
  else if server is Us and stat == 'return': added += 'T'                      // untracked
side-out
  else if server is Them and stat == 'serve': added += 'U'                    // untracked
side-out
  if side == 'out':                 added += 'T'                              // an Out is
always their point
  pending = (side == 'in') ? stat : null
  count += 1; points += added; history.push({ kind:'count', ..., points: added, servedFirstSet:
sfSet })

server = last letter of points ('U' means Us, 'T' means Them), or servedFirst when the log is
empty
```

Companion mockup: <docs/auto-score-mockup.html> plays option D with working buttons and replays the worked example above.